

Agentic Computational Reproducibility for Published Research

CSE598 (Agentic AI) — Capstone Project Proposal

Student name	Jason Lunder
Project title	Agentic Computational Reproducibility of Published Research on CORE-Bench
Repository / notebook link	https://github.com/jlunder00/repro-agent
Configuration location	examples/ and .env.example

1 Problem Definition

Published computational research is often not reproducible: code capsules fail to run months or years later because of dependency rot, undocumented environments, or ambiguous run instructions. Verifying a paper’s numbers requires a human to install the right toolchain, run the code, and cross-check the output — slow work that does not scale. This project builds an *agentic system* that automates it. **Input:** a paper’s code capsule (source plus a README and/or Dockerfile) and questions about its reported results. **Output:** a `report.json` mapping each question to an answer. **Success:** every answer matches ground truth under the benchmark’s grading rule (§4); **failure** is any wrong answer, a crash, or no file produced. The user is a researcher, reviewer, or student confirming that a paper’s claimed numbers actually come out of its code.

2 Motivation and Project Scope

Reproducibility is a load-bearing problem for science, and checking it is exactly the kind of multi-step, tool-using, failure-driven work that motivates an agentic rather than a single-inference approach: the agent must act on the environment, observe what breaks, and revise. We target CORE-Bench [Siegel et al., 2024b,a]: 270 tasks from 90 papers (computer science, social science, medicine; Python and R), split 45 train / 45 test, each posed at three tiers — *easy* (given a successful run’s output, execution forbidden), *medium* (given a Dockerfile plus README), and *hard* (README only; the agent must install dependencies and infer the commands). On the 135-task *test* split the best published agent (CORE-Agent, GPT-4o) reaches only 21.48% on hard [Siegel et al., 2024b], “showing the vast scope for improvement in automating routine scientific tasks.” The capstone goal is an iterative pipeline that improves on that; this proposal covers the *baseline* it will be measured against. **Scope IN:** Python capsules, CPU-only, small capsules (roughly two-thirds of the test split is under 26 MB). **Scope OUT (deferred):** R capsules, GPU tasks, vision/figure questions, multi-gigabyte capsules, Azure parallel execution.

3 Runnable Baseline

The baseline is a deliberately *strict, single-pass, no-retry* pipeline: (1) fetch the capsule, (2) read the README, (3) issue *one* LLM call proposing install and run commands, (4) execute those commands *once* in Docker, (5) issue *one* LLM call to extract answers from the captured stdout, (6) write `report.json`, and (7) score it with the vendored official scorer. No retries, self-correction, or re-planning are permitted at any stage.

The weakness is the point: the baseline is a *scientific control*. The capstone system will add an iterative loop that observes failures, revises commands, and re-reads documentation. Without a strict single-pass floor, any gain from that loop would be confounded with gains from a better model or more prompt engineering; holding model, prompts, and one-shot budget fixed and varying only “iterate or not” isolates the loop’s marginal value. Single-pass execution also attributes every failure to exactly one stage (planning, execution, or extraction), showing where the adaptive system must invest. LLM access is provider-agnostic via `litellm`, so the pipeline runs unmodified with an Anthropic or OpenAI key.

4 Test Case and Baseline Output

The running example is `capsule-9052293` (1.0 MB), an ANP-TOPSIS tourism model. **Input:** the capsule plus the question “Report the closeness coefficient for location L1.” **Expected output:** `report.json` containing 0.844703753651819. All three ground-truth runs are identical, so the prediction interval collapses to a point and an exact match is required. The same capsule is run at both tiers.

```
run_baseline.py - bash
$ export ANTHROPIC_API_KEY=... # or OPENAI_API_KEY
$ python run_baseline.py --capsule capsule-9052293 --tier easy
[1/1] capsule-9052293 (easy) - An ANP-TOPSIS model for tourist destination choice problems
INFO repro_agent.capsules: Removed REPRODUCING.md, environment/, and run scripts for tier=easy at work/capsule-9052293_easy
-> CORRECT 1.7s $0.0009

Task accuracy (easy): 1/1 = 100.0%
Total cost: $0.0009 Total time: 1.7s
Results written to results/baseline_easy_test.json

$ python run_baseline.py --capsule capsule-9052293 --tier hard
[1/1] capsule-9052293 (hard) - An ANP-TOPSIS model for tourist destination choice problems
INFO repro_agent.capsules: Emptied results/ for tier=hard at work/capsule-9052293_hard
INFO repro_agent.capsules: Removed REPRODUCING.md, environment/, and run scripts for tier=hard at work/capsule-9052293_hard
-> INCORRECT 13.5s $0.0053

Task accuracy (hard): 0/1 = 0.0%
Total cost: $0.0053 Total time: 13.5s
Results written to results/baseline_hard_test.json
```

Figure 1: Actual baseline output — the same capsule at the easy tier (correct) and the hard tier (incorrect).

What worked. On the easy tier the baseline is correct on 2/2 capsules (\$0.0034 and 3.0s per task): reading a populated `results/` directory and extracting one number is well within a single-pass pipeline’s reach. The end-to-end path — fetch, tier preparation, LLM extraction, official scoring — is demonstrably working, so a hard-tier zero reflects the task, not a broken harness.

What did not work. On hard the baseline scores 0/2, both failing at execution. Its one planning call proposed `pip install openpyxl pandas && python code/script.py`; with no retry the run died and the pipeline reported `null`. Following the error chain by hand shows the capsule *is* reproducible, but only via three sequential fixes, each discoverable solely by observing the previous failure: (i) `ModuleNotFoundError: xlrld`; (ii) then `FileNotFoundError`, as the script uses paths relative to `code/`; (iii) then `XLRDError: Excel xlsx file; not supported`, because `xlrld 2.x` dropped `.xlsx` support — requiring `xlrld==1.2.0`, i.e. an *older* version of the package just installed. With all three, the capsule reproduces 0.844703753651819 exactly. That is precisely the observe-and-revise behaviour the control forbids, and the gap the capstone closes.

5 Reproducibility and Run Instructions

`pip install -r requirements.txt`, then copy `.env.example` to `.env` and set exactly one of `ANTHROPIC_API_KEY` or `OPENAI_API_KEY`; capsule metadata and prompt templates live under `examples/`. Docker is required only for `--tier hard`. A single task is run with `python run_baseline.py --capsule capsule-9052293 --tier easy`, which fetches the capsule (1.0MB, cached in `capsules/`), runs the pipeline of §3, and writes results to `results/`. `--list` previews the selection and capsule sizes without downloading or needing a key; `--subset`, `--tier`, and `--limit` run batches. The easy-tier example above completes in under two seconds for well under a cent.

6 Initial Evaluation Plan

We will compare the future iterative agentic system against this single-pass baseline on: (i) official task accuracy (all-or-nothing per task), (ii) per-question accuracy, (iii) wall-clock latency, (iv) dollar cost per task, and (v) a failure-stage breakdown (install vs. execution vs. extraction) to show *where* iteration recovers failures the baseline cannot. Cost matters: published successful tasks averaged \$0.54 versus \$2.59 for failures, so failure costs more than success and early failure detection pays. The published numbers below (test split, 45 tasks/tier) are our external reference point; the benchmark reports no human/expert baseline.

Agent	Model	Easy	Medium	Hard
CORE-Agent	GPT-4o	60.00%	57.78%	21.48%
CORE-Agent	GPT-4o-mini	44.44%	32.59%	16.30%
AutoGPT	GPT-4o	35.56%	37.78%	6.67%
AutoGPT	GPT-4o-mini	8.89%	2.22%	2.22%

Siegel et al. [2024b], Table 5 (pass@1, test split, 45 papers/tier).

Measured baseline (2 Python capsules, `claude-sonnet-5`): **easy 2/2 = 100%** task and per-question accuracy, \$0.0034 and 3.0s per task; **hard 0/2 = 0%**, \$0.0049 and 9.7s per task, with **2/2 failures at the execution stage** and none at planning or extraction. The medium tier is not implemented (it needs Docker-in-Docker). The execution-stage concentration is the actionable result: it says the iterative loop should target dependency and invocation repair first, not answer extraction.

7 Limitations and Next Steps

Dependency rot is the central difficulty, not an incidental one: as §4 shows, the fix can be to install an *older* package version, which no amount of up-front planning recovers. *Sample size*: two capsules is a demonstration, not an estimate. *Resource and language scope*: GPU and multi-gigabyte capsules are excluded, and R capsules (about half of CORE-Bench) are deferred, so results characterize only the Python subset. *Retrieval caveat*: the easy tier currently concatenates result files in directory order until a character budget is exhausted, so an answer buried in a long log can be truncated away. *Stochastic scoring*: with a 95% prediction interval, a faithfully reproduced run can fall outside by chance, so even a perfect agent would not score 100%. Next: extend to the full Python/CPU test subset, implement the medium tier, add the iterative loop targeting execution-stage failures first, and evaluate question-guided result retrieval.

References

- Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. CORE-Bench GitHub repository. <https://github.com/siegelz/core-bench>, 2024a. Accessed 2026-09-06.
- Zachary S. Siegel, Sayash Kapoor, Nitya Nadgir, Benedikt Stroebl, and Arvind Narayanan. CORE-Bench: Fostering the credibility of published research through a computational reproducibility agent benchmark. *arXiv preprint arXiv:2409.11363*, 2024b. URL <https://arxiv.org/abs/2409.11363>.